2D Single-Target Window-Measure Note

The Difference Between Occupancy Share and Ever-Visit Probability

Main Point

This note now uses a milder corridor-only variant to explain the same question: why `target funnel` is not at 100%, even though every trajectory must eventually pass through it.

The answer has two parts:

1. The current figure shows **window-conditioned occupancy share**: how much pre-hit time is spent in each region.
2. The intuitive statement "every trajectory must pass the funnel" corresponds to **ever-visit probability**: whether the trajectory ever enters that region before absorption.

Both are correct, but they are different observables and therefore need not have the same numerical values.

1 Geometry and Spatial Partition

We keep the same symmetric one-target geometry from `grid2d_one_two_target_gating`, but switch to a milder corridor-only soft-bias variant:

$$L_x = 60, \qquad W_y = 15, \qquad (x_s, y_s) = (7, 7), \qquad (x_t, y_t) = (58, 7).$$

The corridor band is $y = 5, \ldots, 9$. The two semipermeable membranes sit on the upper and lower interfaces between the corridor and the outer reservoir, and the membrane span is $x = 5, \ldots, 54$. In this explainer we do not draw the gate contour; we keep only the spatial partition. To make the color-to-region mapping immediately readable, both main figures below repeat the same partition schematic directly above the corresponding plot instead of forcing the reader to look back.

The corridor-related dynamical parameters of this baseline are also fixed and should be stated explicitly:

- the corridor half-width is 2, so the midline is $y = 7$ and the corridor is exactly $y = 5, \ldots, 9$;
- the wall margin is 5, so the semipermeable membranes only cover $x = 5, \ldots, 54$, i.e. the interface edges $(x, 4) \leftrightarrow (x, 5)$ and $(x, 9) \leftrightarrow (x, 10)$;
- every corridor site carries a **local eastward bias**, with direction `E`;
- inside the membrane span, namely for $5 \leq x \leq 54$, the local bias strength is reduced to $\delta_{\text{core}} = 0.8$;
- in the left and right corridor openings, namely for $x < 5$ or $x > 54$, the local bias strength is $\delta_{\text{open}} = 0.0$, so the openings no longer receive any extra rightward push;
- the upper and lower outer reservoirs already had no local rightward bias; after this change, the only sites with local rightward driving are the main corridor cells $5 \leq x \leq 54,\ 5 \leq y \leq 9$;
- the variant still includes a **global leftward bias** $(b_x, 0) = (-0.08, 0)$, so outside the main corridor segment the dynamics reduce to that weaker leftward background drift;
- the symmetric membrane permeabilities are $kappa_{c\to o} = kappa_{o\to c} = 0.002$;
- the laziness parameter used in the transition construction is $q = 0.8$.

One subtle but important point is that the main-corridor local bias points to the *right*, whereas the global bias points to the *left*; the bimodal structure is produced by the competition between these two driving layers together with the membrane bottleneck. We also ran an exact scan with $\delta_{\text{open}} = 0$: once $\delta_{\text{core}} \leq 0.6$ the two-peak structure collapses, while $\delta_{\text{core}} = 0.8$ is a reasonable compromise between "not too hard" and "still visibly bimodal".

2 How Peak1, Valley, and Peak2 Are Defined

These names are not chosen by eye. They come from the repo-native exact rules.

2.1 Step 1: detect two credible peaks

Given the total first-passage pmf $f(t)$, the code first builds a smoothed envelope with window length 7, then calls `find_two_peaks` to detect two credible local peaks. The routine does not merely ask for local maxima. It also requires:

- each peak height to be at least 6% of the global maximum;
- a minimum separation of 20 time steps between the two peaks;
- a genuine valley between the peaks rather than a shoulder;
- no extreme peak imbalance;
- a visible valley drop relative to the peak heights.

So `peak1` and `peak2` are the two peak locations that survive this credibility filter.

2.2 Step 2: define the valley between them

Once two credible peaks are found, the code takes the minimum of the smoothed curve between them:

$$t_{\text{valley}} = \arg \min_{t \in [t_{\text{peak1}},\, t_{\text{peak2}}]} f_s(t),$$

where f_s denotes the smoothed curve. Hence:

- the center of `peak1` is t_{peak1};
- the center of `valley` is t_{valley};
- the center of `peak2` is t_{peak2}.

2.3 Step 3: choose a shared window half-width

The windows are not hand-tuned. Their common half-width is

$$\text{half} = \max\left(8,\ \min\left(26,\ \left\lfloor \frac{t_{\text{peak2}} - t_{\text{peak1}}}{7} \right\rfloor\right)\right).$$

The three windows are then

$$[t_{\text{peak1}} - \text{half},\ t_{\text{peak1}} + \text{half}], \quad [t_{\text{valley}} - \text{half},\ t_{\text{valley}} + \text{half}], \quad [t_{\text{peak2}} - \text{half},\ t_{\text{peak2}} + \text{half}].$$

For the current corridor-only soft variant, the exact centers are

$$t_{\text{peak1}} = 554, \qquad t_{\text{valley}} = 1490, \qquad t_{\text{peak2}} = 2327.$$

Since $t_{\text{peak2}} - t_{\text{peak1}} = 1773$, this still yields half $= 26$, so the actual windows used in this report are

$$\texttt{peak1} = [528, 580], \qquad \texttt{valley} = [1464, 1516], \qquad \texttt{peak2} = [2301, 2353].$$

2.4 Relation to the phase criterion

The window definition is not the same thing as the clear-double-peak phase criterion. The windows only define three representative time ranges. The phase test separately checks whether the two peaks are truly separated. In the canonical one-target rule:

- `find_two_peaks` must identify two credible peaks;
- the separation score must satisfy $\mathrm{sep}_{\mathrm{peaks}} \geq 1$ for a case to count as a clear double peak.

For the present soft-bias variant we obtain $\mathrm{sep}_{\mathrm{peaks}} \approx 0.675$. So the case still has two credible peaks, but it no longer qualifies as a *clear* double peak under the repo-native threshold; in phase language it is `phase=1` rather than `phase=2`.

3 What the Current Figure Actually Measures

3.1 Window-conditioned occupancy share

Let X_t be the position at time t, let T be the first-passage time, and let W be one of the windows `peak1`, `valley`, or `peak2`. For any region R, the main quantity shown in the current figure is

$$p_W^{\mathrm{occ}}(R) = \frac{\mathbb{E}\Big[\mathbf{1}_{\{T\in W\}} \sum_{t=0}^{T-1} \mathbf{1}_{\{X_t\in R\}}\Big]}{\mathbb{E}\big[\mathbf{1}_{\{T\in W\}}\, T\big]}.$$

Its meaning is:

> Among trajectories that hit in window W, pick a random pre-hit time point; the probability that it lies in region R is $p_W^{\mathrm{occ}}(R)$.

This is therefore a **time-weighted** quantity. A trajectory may be forced to pass through the funnel, but if it spends very little time there, its occupancy contribution remains small.

3.2 Why the funnel does not automatically become 100%

Consider a toy example:

- every trajectory spends about 100 steps in the corridor;
- then it enters the funnel and spends only 2 steps there before absorption.

Then

$$p^{\mathrm{visit}}(\texttt{funnel}) = 100\%, \qquad p^{\mathrm{occ}}(\texttt{funnel}) \approx \frac{2}{102} \approx 2\%.$$

So "must pass through" does not imply "dominates the time-weighted occupancy".

4 What the Intuitive Question Refers To

4.1 Ever-visit probability

If instead we ask:

> Among trajectories that hit in window W, how many ever enter region R before absorption?

then the relevant quantity is

$$p_W^{\mathrm{visit}}(R) = \mathbb{P}(\exists t < T:\ X_t \in R \,|\, T \in W)\,.$$

This is a path-event probability, not a time-occupancy fraction.

For `target funnel` in the present geometry, the exact calculation gives

$$p_{\texttt{peak1}}^{\mathrm{visit}}(\texttt{funnel}) = p_{\texttt{valley}}^{\mathrm{visit}}(\texttt{funnel}) = p_{\texttt{peak2}}^{\mathrm{visit}}(\texttt{funnel}) = 1.$$

So the intuition that every trajectory must pass through the funnel is correct; it simply corresponds to **ever-visit**, not to **occupancy share**.

5 Direct Comparison of the Two Observables

We keep both of the following figures as main figures rather than treating one as mere supplementary material, and we place them back to back so the reading flow is not interrupted by other plots or tables. A good reading order is: first inspect Fig. 1 to understand where the time is spent; then move directly to Fig. 2 to see how the ranking changes when we switch from a time-weighted observable to a path-event observable.

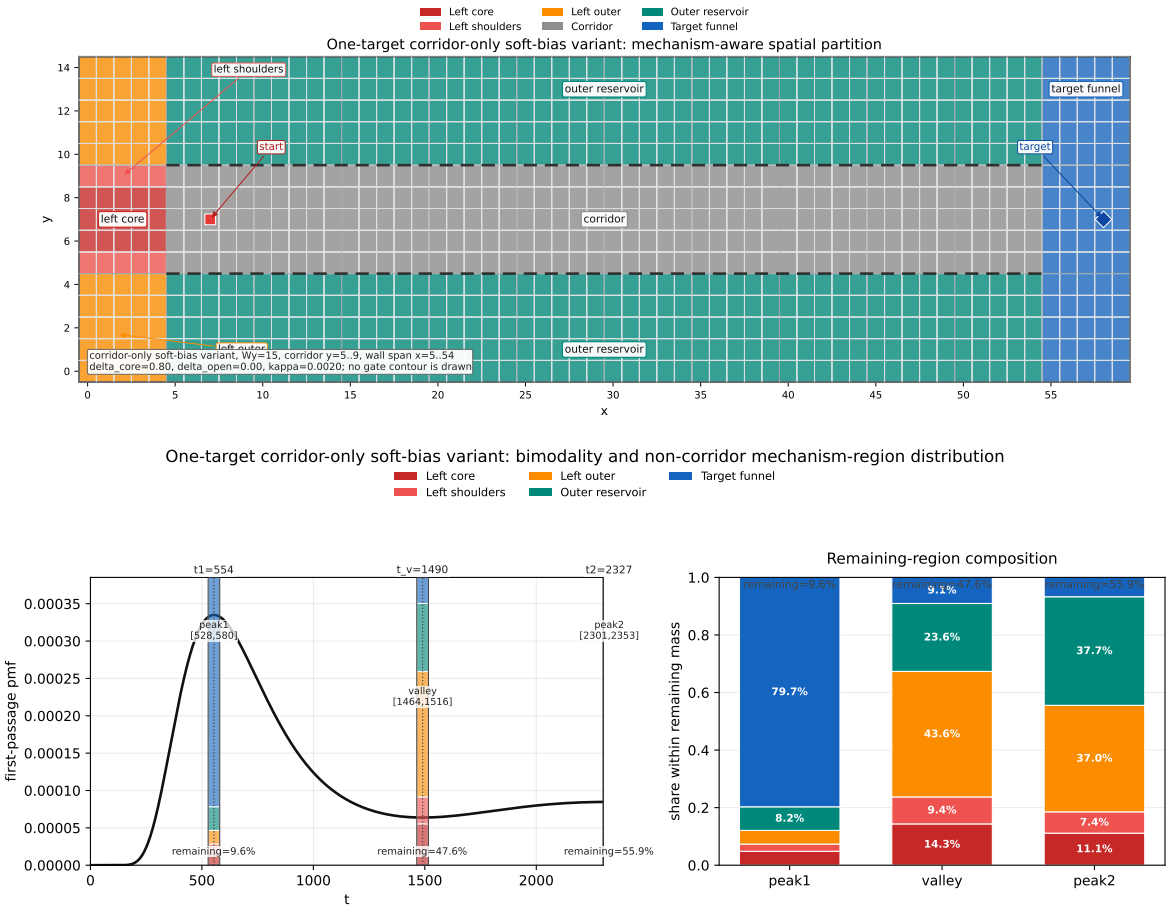

Figure 1: Overview layout for the first main figure. Top: mechanism-aware spatial partition of the corridor-only soft-bias variant, with the full state space split into `left core`, `left shoulders`, `left outer`, `corridor`, `outer reservoir`, and `target funnel`. Bottom: the occupancy main figure, where full-height window-composition bars are drawn directly on top of the exact bimodal first-passage curve. The right panel shows the same non-corridor composition as standalone stacked bars. This keeps the color-to-region mapping visible right next to the figure.

Read Fig. 1 first as the "time-budget" view: it tells us where the pre-hit time mass accumulates within the same hitting window once the corridor has been factored out.

Region	Occ. p1	Visit p1	Occ. valley	Visit valley	Occ. p2	Visit p2
Target funnel	7.7%	100.0%	4.3%	100.0%	3.8%	100.0%
Outer reservoir	0.8%	21.9%	11.2%	93.5%	21.1%	99.9%
Left outer	0.5%	6.9%	20.7%	87.2%	20.7%	99.3%
Left shoulders	0.2%	13.4%	4.5%	88.4%	4.1%	99.3%
Left core	0.5%	16.9%	6.8%	86.7%	6.2%	97.9%

Table 1: Exact comparison between occupancy share and ever-visit probability across the three windows.

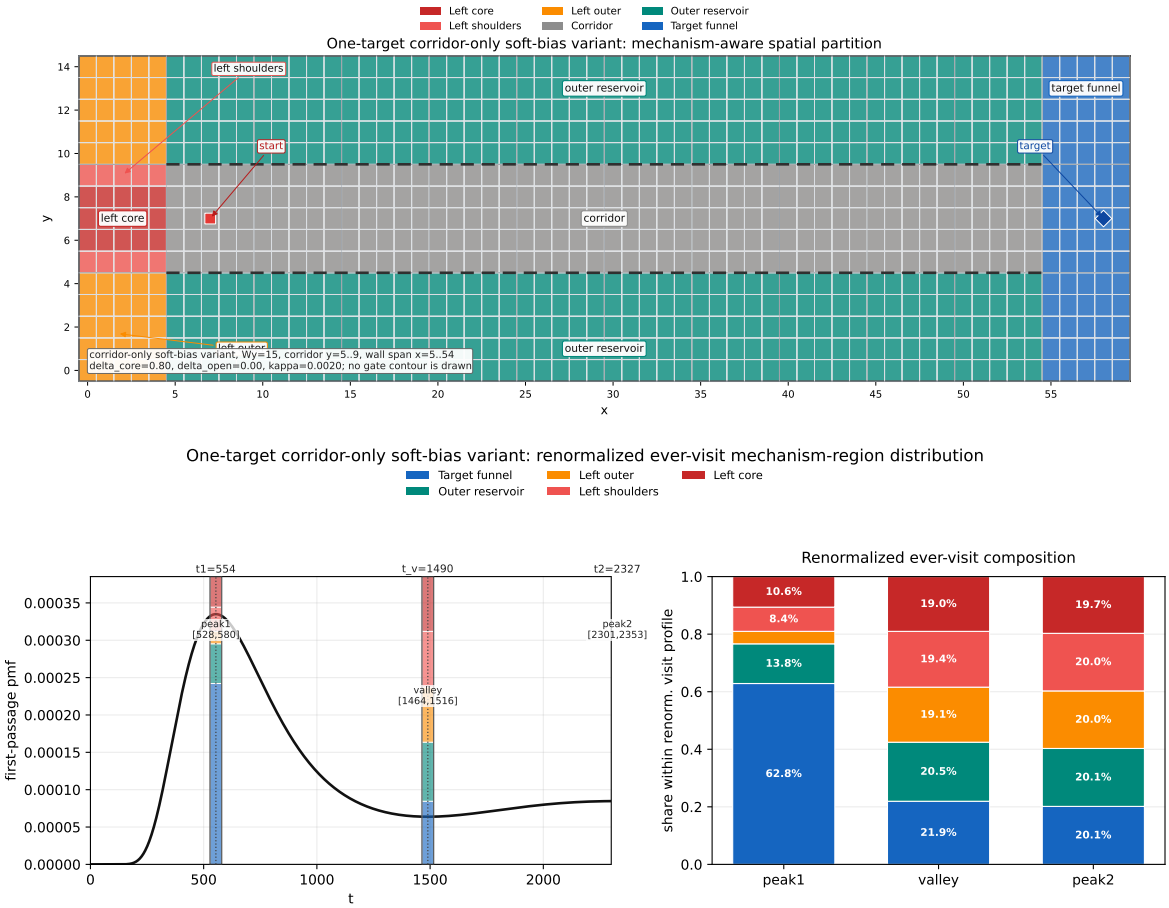

Figure 2: The second main figure uses the same overview layout, with the spatial partition placed directly above the ever-visit figure. The lower panel keeps the same exact bimodal first-passage curve and the same three windows, but replaces the regional weights by the normalized ever-visit profile. The right panel shows the renormalized ever-visit composition for each window. This is not a direct stack of raw ever-visit probabilities; the normalization is used only to make the shape comparable to the occupancy figure within the same visual grammar.

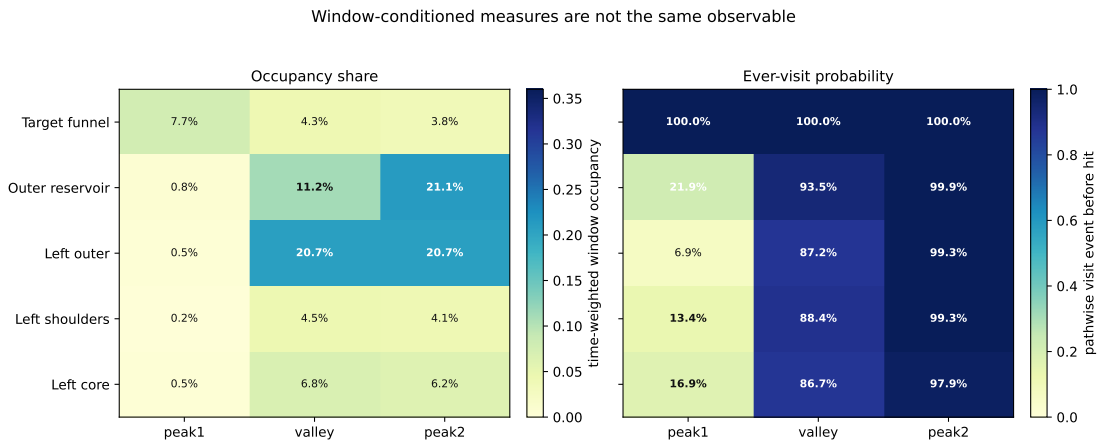

Figure 3: Occupancy share on the left and ever-visit probability on the right. The key row is `target funnel`: its occupancy share is only about 7.7%/4.3%/3.8%, while its ever-visit probability is 100% in all three windows. This is exactly the distinction between a time-weighted observable and a path-event observable.

Window	Cosine similarity	Total variation distance
peak1	0.9838	0.1728
valley	0.8270	0.2758
peak2	0.8153	0.3461

Table 2: Similarity between the two normalized profiles on the same five-region basis. A cosine similarity closer to 1 means the shapes are more alike; a smaller total-variation distance means they are closer in mass distribution.

It is important that Table 2 compares the shapes of two *normalized profiles*. It does *not* mean that raw ever-visit probabilities themselves form a composition. The comparison only answers whether the relative importance ordering of regions is similar across the two observables.

6 Why Monte Carlo and Exact Computation Agree

6.1 If Monte Carlo estimates occupancy share

Let the m-th Monte Carlo trajectory have first-passage time T_m and path $(X_t^{(m)})_{t=0}^{T_m-1}$. The natural Monte Carlo estimator for occupancy share is

$$\widehat{p}_W^{\mathrm{occ}}(R) = \frac{\sum_{m=1}^{M} \mathbf{1}_{\{T_m \in W\}} \sum_{t=0}^{T_m-1} \mathbf{1}_{\{X_t^{(m)} \in R\}}}{\sum_{m=1}^{M} \mathbf{1}_{\{T_m \in W\}} T_m}.$$

This is exactly "flatten all pre-hit time points from trajectories that hit in window W and count what fraction falls in R". By the law of large numbers, it converges to the exact occupancy share shown in the figure.

6.2 If Monte Carlo estimates ever-visit probability

The corresponding estimator for ever-visit probability is

$$\widehat{p}_W^{\mathrm{visit}}(R) = \frac{\sum_{m=1}^{M} \mathbf{1}_{\{T_m \in W\}} \mathbf{1}_{\{\exists t < T_m:\ X_t^{(m)} \in R\}}}{\sum_{m=1}^{M} \mathbf{1}_{\{T_m \in W\}}}.$$

This converges to the exact ever-visit probability, not to occupancy share.

6.3 So when should Monte Carlo and exact results look similar?

The answer is simple:

- if Monte Carlo estimates the same observable, it should agree with the exact result up to sampling noise;
- if Monte Carlo estimates a different observable, it may look very different.

In the present example:

- a time-point-based Monte Carlo estimate converges to occupancy share;
- an event-based Monte Carlo estimate gives `target funnel` close to 100%.

Summary

The main purpose of this note is not to introduce a new mechanism, but to separate two observables that are often mixed together:

1. **occupancy share**: time-weighted, asking where the process spends its time;
2. **ever-visit probability**: event-based, asking whether the process ever reaches a region.

Once the distinction is made explicit, there is no contradiction in saying that `target funnel` is a mandatory path event while still carrying only a small fraction of the time-weighted spatial distribution.